Yük testi patladığında kök nedeni dakikalar içinde bulmak

Loki + Grafana MCP ile AI destekli triage

Laboratuvar sonuçları ve kurumsal pilot önerisi

6 Kasım 2026

Yönetici özeti

Problem

Yük testi eşiği aşınca test durur; 'neden' sorusu saatlerce elle log aramasıyla cevaplanır.

Çözüm

Servis loglarını Loki'de toplayıp AI istemcisinin Grafana MCP üzerinden salt-okunur sorgulamasına açtık. Tek talimat dosyası, tek girdi: zaman penceresi.

Sonuç

Laboratuvarda bilerek yerleştirilen iki farklı kök nedeni (kapasite sınırı ve kod hatası) AI kanıtlarıyla doğru ayırdı; test aracına ve istemciye bağımlı değil.

2 / 2

kök neden doğru ayrıştırıldı, kanıtlı

~5 dk

rapor süresi (pencere verilmeden, otomatik)

0

test aracına / AI istemcisine bağımlılık

Bugün yük testi patlayınca ne oluyor

Test ekibi 'eşik aşıldı' der

Rapor test aracının çıktısıdır: hata oranı ve gecikme. Hangi kod yolunun patladığı yazmaz.

Loglar dağınık, arama elle

Kibana/Grafana'da saatlerce filtre; hangi hata hangi isteğe ait, trace olmadan belirsiz.

Yük mü, kod mu? sezgiye kalıyor

Havuz tükenmesiyle null-pointer aynı anda görünür; biri kapasite, diğeri bug. Karıştırmak yanlış düzeltmeye yol açar.

Test tekrarı gecikir

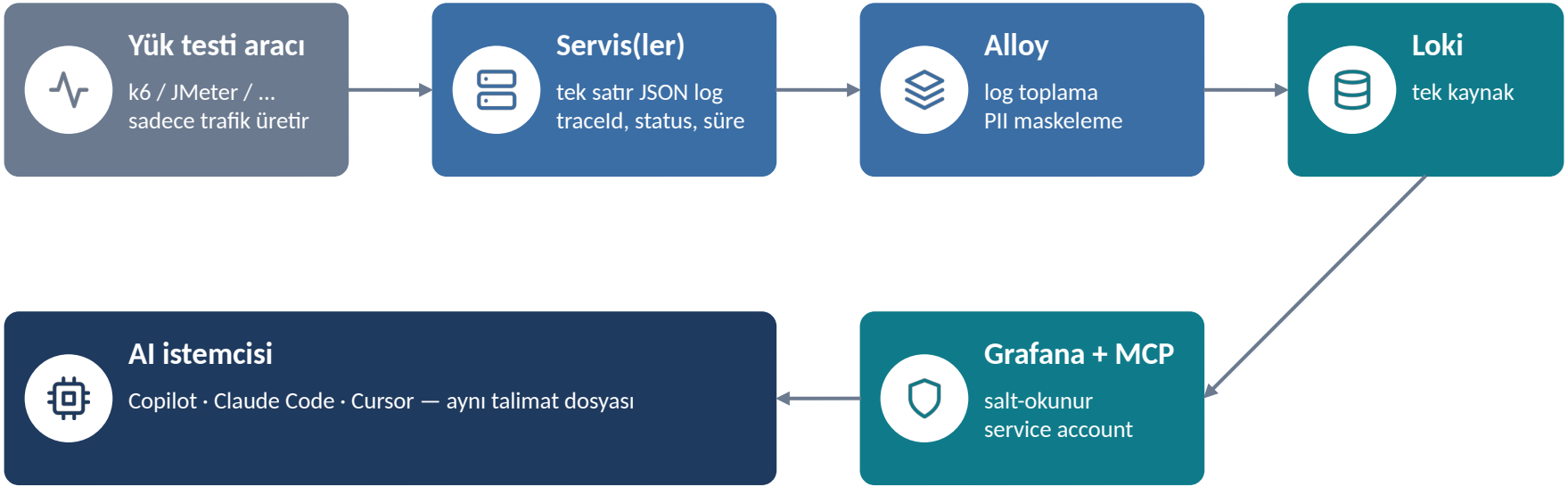
Kök neden netleşene kadar bir sonraki koşu planlanamaz; her tur günler sürer.

Hedef

Test ekibinden yalnızca 'test şu saatte koştu' bilgisini alıp, kod ekibine kanıtlı bir kök neden raporu vermek.

Kod erişimi, test script'i veya özel araç entegrasyonu gerekmeden.

Çözüm: tek veri kaynağı Loki, AI yalnızca sorgular



Tempo veya Prometheus yok. Trace korelasyonu, her log satırındaki traceId ile yapılır; yüksek kardinaliteli alanlar Loki'de label değil, satır içinde kalır.

Triage nasıl çalışır: tek talimat dosyası, sekiz adım

0 Pencereyi bul

Trafik hacminin sıçradığı aralığı sorgula, onaylat

1 Hacim ve hata şekli

Dakikalık istek, 5xx, p95 serileri

2 5xx'i grupla

endpoint × status tablosu

3 Hataları kümele

exception sınıfı + mesaj

4 traceId izle

Küme başına 3 trace, tüm satırlar

5 Servisler arası

Kök neden downstream mı, upstream mı

6 Yük mü, kod mu

Hata oranı yükle artıyor mu, sabit mi

7 Rapor

Özet · Kök neden · Kanıt · Etki · Düzeltme · Devam

Laboratuvar: iki bilerek yerleştirilmiş hata

A · Kapasite sınırı (yük kaynaklı)

- Skorlama servisinde 10 bağlantılık havuz
- Gecikme eşzamanlılıkla büyür, 2 sn bekleyen istek 503 alır
- Beklenen imza: hata oranı istek sayısıyla birlikte artar

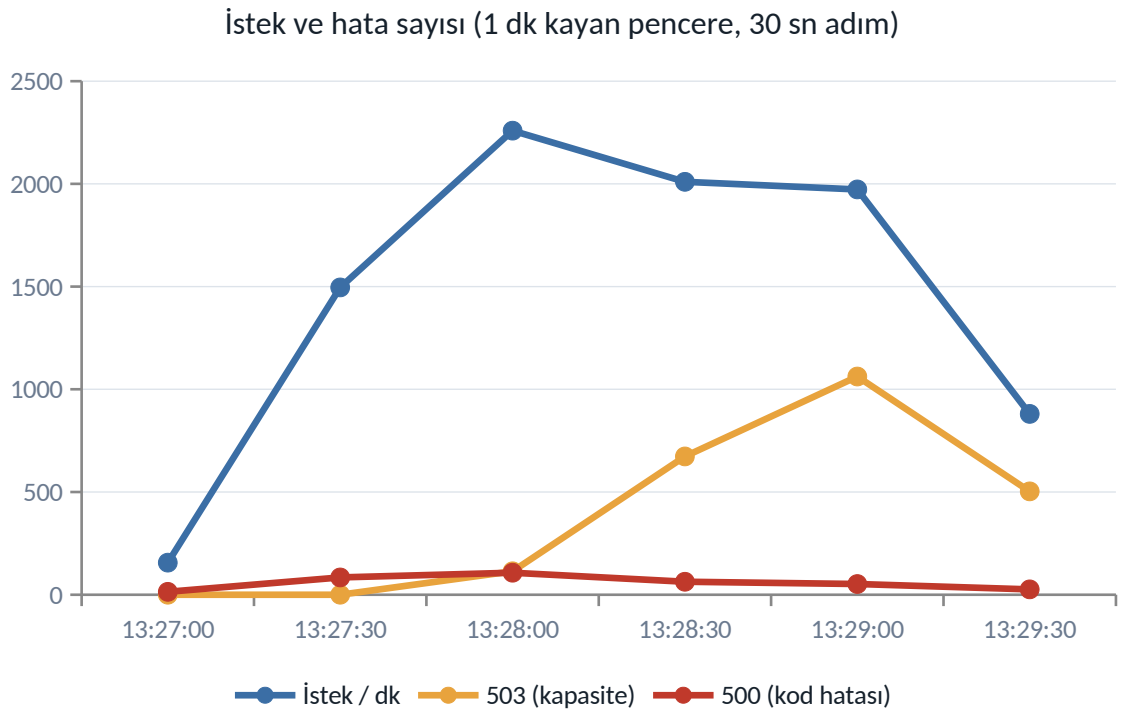
B · Kod hatası (yükten bağımsız)

- Kampanya kodu gelmeyen uzun vadeli başvuruda null referans
- Global hata yakalayıcı 500 döner, stack trace loglanır
- Beklenen imza: isteklerin sabit bir yüzdesi, trafikten bağımsız

Yük profili

0 → 20 → 80 sanal kullanıcı (30 s + 60 s rampa, 60 s sabit). Eşikler: hata oranı < %5 (aşılınca test durur), p95 < 800 ms. İsteklerin %10'u kampanya kodsuz gönderilir.

Sonuç 1: Test beklendiği gibi patladı



Yük testi aracının verdiği

%30

başarısız istek (eşik %5)

3,09 s

p95 gecikme (eşik 0,8 s)

2 dk 02 sn

sonra test kendini durdurdu

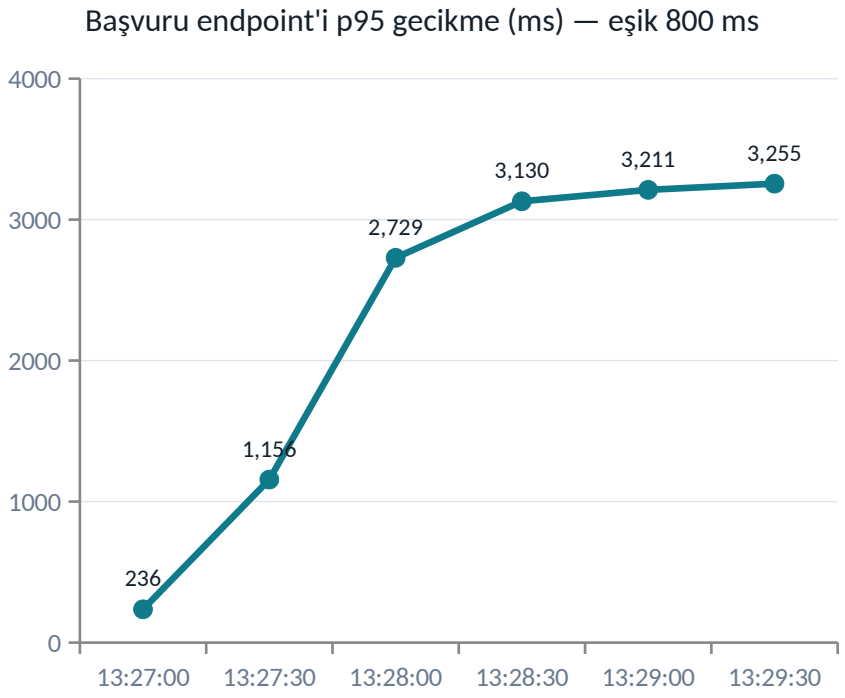
Sonuç 2: AI iki kök nedeni doğru ayırdı

Kapasite sınırı — yük kaynaklı

503 oranı 1.738 istek/dk'da %0, 2.100 istek/dk'da %58. p95 gecikme 238 ms'den 3,2 s'ye. Mesaj: 'scoring pool exhausted'.

Null referans — kod hatası

500 oranı her dakikada %8–12, trafikten ve gecikmeden bağımsız; testin ilk saniyelerinden itibaren var. Stack trace tek satıra işaret ediyor.



Sonuç 3: Aynı traceId iki serviste

Trace A · 503

scoring-service	`ERROR pool exhausted, inflight=41`
scoring-service	`503 POST /scores  2022 ms`
credit-service	`503 POST ../shopping/apply 2038 ms`

→ Kök neden downstream'de (skorlama servisi); credit-service yalnızca yansıtıyor.

Trace B · 500

scoring-service	`200 POST /scores  202 ms`
credit-service	`ERROR NPE InstallmentCalculator:39`
credit-service	`500 POST ../shopping/apply 237 ms`

→ Kök neden upstream'de (kredi servisi); skorlama temiz.

1.171 credit-service 503 = **1.171** scoring-service 503 = **1.171** 'pool exhausted' ERROR satırı

AI'nın çıkarımı: arada zaman aşımı veya bağlantı hatası yok

Rapor neye benziyor

Özet
Pencere, toplam istek, 5xx, kaç kök neden

Kök neden(ler)
Ad, tür (yük / kod), ilk görülme zamanı

Kanıt
LogQL sorguları, traceId'li örnek satırlar, dakikalık tablo

Etkilenen endpoint ve oran
endpoint × status × adet

Önerilen düzeltme
Sınıf ve satır düzeyinde somut öneri

Testi devam ettirme
Hangi düzeltmeden sonra hangi eşik izlenmeli

Rapordan alıntı (AI çıktısı, kısaltıldı)

Kök neden 1 — yük kaynaklı — downstream (scoring-service)
Havuz 10; eşzamanlı istek 34–46'ya çıkınca 2000 ms bekleyip 503 dönüyor. İlk görülme 14:06:02Z. credit-service'teki 1.171 adet 503, scoring-service'teki 1.171 ERROR ile birebir eşleşiyor.

Kök neden 2 — kod bug'ı — upstream (credit-service)
InstallmentCalculator.resolveRate içinde campaign null iken getRate() çağrılıyor. Oran %8–12, yükten bağımsız. İlk görülme 14:05:11Z, testin ilk saniyeleri.

Not: query_loki_stats bu pencere için sıfır döndü; satır sayıları metrik sorgularından alındı, stats çıktısı doğrulanmadı.

Kurumsal ortam için hazır olanlar

Kişisel veri maskeleme

Müşteri tanımlayıcıları Loki'ye girmeden maskelenir; rapor traceId ile referans verir.

Salt-okunur, yalnızca Loki

Viewer service account; MCP sunucusunda yalnızca 9 okuma aracı açık, yazma yok.

İstemciden bağımsız

Copilot, Claude Code, Cursor aynı talimat dosyasını kullanır; yeni istemci = bir konfig dosyası.

UAT ve testten bağımsız

Tek girdi zaman penceresi. UAT'ye, test script'ine veya sonuç dosyasına erişim gerekmez; AI yalnızca Grafana API'sine gider.

Docker'sız çalışma

MCP sunucusu binary olarak da çalışır; proxy ve kurumsal sertifika geçişi var.

Log sözleşmesi

Gerekli alanlar dokümante; .NET (Serilog) örneği aynı sorgularla doğrulandı.

Sınırlar ve riskler (açıkça)

Label ve alan adları platform ekibine bağlı

Talimat dosyasındaki eşleme tablosu kurum değerleriyle doldurulur; servis koduna dokunmadan uyarlanır.

Çok satırlı stack trace

JSON loglamayan servislerde satırlar dağılır; log hattında birleştirme veya servis tarafında JSON encoder gerekir.

Loki sorgu limitleri

Kurumsal Loki'de satır ve süre sınırları var; şablonlar seçici ayrıştırma ve küçük limitlerle yazıldı.

Yapay zekâ erişim politikası

Hangi istemcinin onaylı olduğu ve log verisinin modele gidip gidemeyeceği bilgi güvenliğiyle netleşmeli.

Rapor insan doğrulaması ister

AI kanıt gösterir ama kararı vermez; her rapor kod ve test ekibince teyit edilir.

Pilot önerisi: üç adım, altı hafta

1 Hafta 1–2 · Hazırlık

- Bir servis log sözleşmesine uydurulur
- Grafana'da salt-okunur service account
- Bilgi güvenliği onayı: maskeleme + istemci

2 Hafta 3–4 · İlk gerçek triage

- Test ekibi normal yük testini koşar
- Bize yalnızca zaman penceresi iletilir
- AI raporu kod ekibine verilir

3 Hafta 5–6 · Değerlendirme

- Kök neden isabeti (ekip teyidi)
- Triage süresi: elle vs AI
- Devam / genişlet / durdur kararı

İstenen karar

Pilot onayı, bir aday servis ve test ekibinden bir yük testi penceresi. Yeni altyapı satın alımı yok; mevcut Grafana/Loki ve onaylı AI istemcisi yeterli.

Test ekibinden bir zaman penceresi, kod ekibine kanıtlı bir kök neden

Laboratuvar deposu: README, triage/LOADTEST_TRIAGE.md, triage/reports/ altındaki iki AI raporu

Sorular ve pilot için aday servis önerileri